PX Practice papers: answers

Model answers sized to the marks, in the same order as the question papers.

INTE 22313
4 × 25 MARKS

P1 P2 P3

What these papers are built on

- **All four questions are case-based**, and a single case asks about **two or more patterns**, spread across the whole question rather than saved for the last part (exam briefing).
- The shape of the past papers: identify the patterns with reasons → diagram → short Java → a follow-up part (challenges, HA/DR, SOLID, how two patterns connect).
- Question 2 is microservices in every paper, because 2023, 2024 and 2025 all had it.
- Scenarios the past papers haven't used yet: food delivery, smart home, online exams, ride-hailing, insurance, banking, EV charging, airline, hotel, returns.

How to mark yourself

1. **Pattern named correctly** with its group: about a third of the marks for an "identify" part.
2. **Reasons tied to the case** (not general theory): most of the rest. "Each state is a class, so pay() is refused in Idle" scores; "State is a behavioural pattern from GoF" alone does not.
3. **Diagrams:** correct arrows, stereotypes and +/−/# matter more than neatness.
4. **Code:** the exact interface and method names from the question, and it must compile in spirit.
5. Where another pattern is reasonable, the answer says so. If you chose it and justified it from the case, give yourself the marks.

The tab numbers in the answers (e.g. "note tab 06") point to the printed exam note, so you can see where the same answer lives during the real exam.

1 Paper 1 · Question 1

Food delivery app

PRACTICE PAPER 1
25 MARKS

PRACTICE PAPER 1

INTE 22313 – Software Design Patterns and Frameworks · No. of questions: 4 · Time: 2½ hours
Answer ALL questions. Allocate time by the marks. State any assumptions you make. This is an OPEN BOOK exam.

QUESTION 1 · 25 MARKS

QuickBite is a food delivery app. A customer's order moves through these stages: **Placed** → **Confirmed** by the restaurant → **Preparing** → **Out for delivery** → **Delivered**. An order can be **Cancelled** only before it is out for delivery. What a customer can do (cancel, track, rate) depends on the current stage.

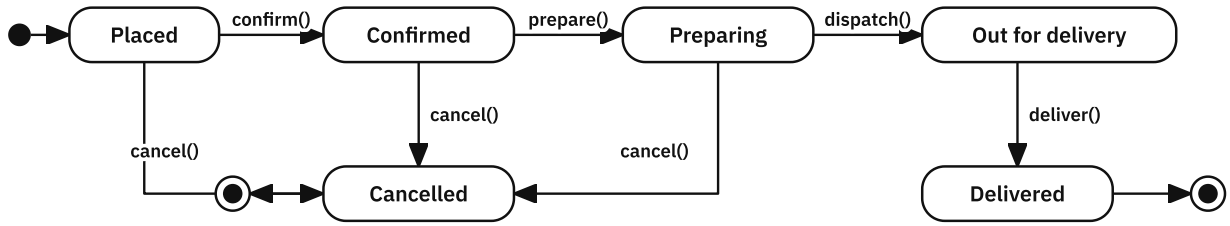
QuickBite works with many restaurants. Each restaurant already uses its own point-of-sale system from a different vendor: one accepts orders through `pushOrder(xml)`, another through `createTicket(json)`. QuickBite cannot change these systems.

When a customer taps "Place order", the app must check the restaurant, take the payment, assign a rider and send notifications. The mobile team wants one simple call for this instead of talking to four services.

1.1 *Identify a suitable design pattern, with reasons, for each of: (a) the order stages, (b) sending orders to the restaurants' different systems, (c) the single "Place order" call.* — 6 marks

(a) order stages	**State**	The order behaves differently in each stage: cancel is allowed while Placed, Confirmed or Preparing but refused once out for delivery; rating only works after Delivered. Each stage becomes a class that knows its allowed actions and its next stage, so there are no if/else checks on a status field.
(b) restaurant systems	**Adapter**	The restaurants' systems have incompatible methods and can't be changed. One adapter per vendor converts QuickBite's `sendOrder(order)` into `pushOrder(xml)` or `createTicket(json)`; a new vendor is just a new adapter.
(c) one "Place order"	**Facade**	`OrderFacade.placeOrder()` calls restaurant check → payment → rider → notification in the right order, so the mobile app depends on one class instead of four services.

1.2 *Draw a state diagram for an order.* — 5 marks



Marks: start dot, every state, each arrow labelled with its event, cancel only from the first three, end states shown.

1 Paper 1 · Question 1 (cont.)

continued

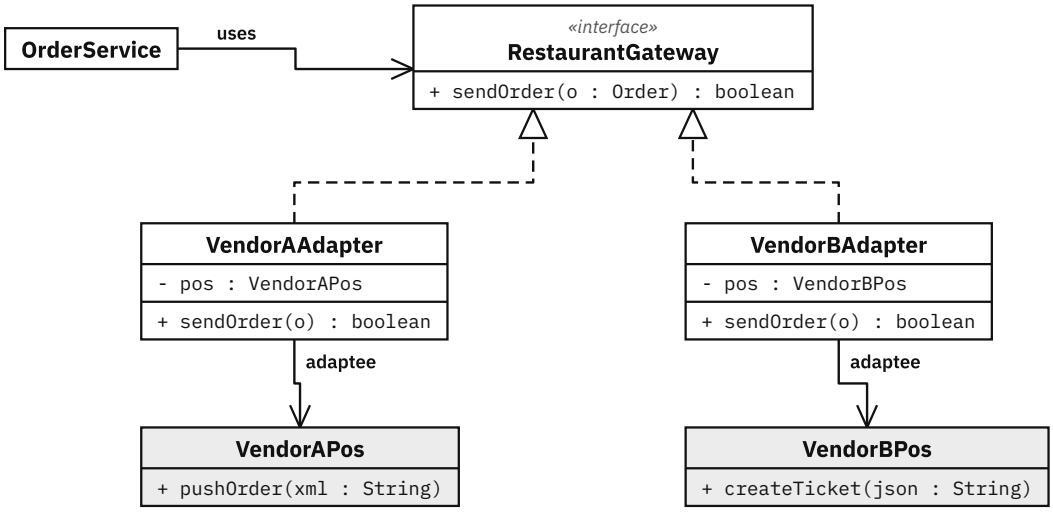
PRACTICE PAPER 1
25 MARKS

1.3 *Write Java for the OrderState interface and the Order context class. Show one concrete state.* 6 marks

```
public interface OrderState {
  void confirm(Order o);
  void startPreparing(Order o);
  void dispatch(Order o);
  void deliver(Order o);
  void cancel(Order o);
}
public class Order {           // context
  private OrderState state =
      new PlacedState();
  public void setState(OrderState s) {
    state = s; }
  // every request goes to the state
  public void confirm() {
    state.confirm(this); }
  public void dispatch() {
    state.dispatch(this); }
  public void cancel() {
    state.cancel(this); }
  // startPreparing(), deliver(): same
}
```

```
public class OutForDeliveryState
        implements OrderState {
  public void deliver(Order o) {
    o.setState(new DeliveredState()); }
  public void cancel(Order o) {
    System.out.println(
      "Too late to cancel"); }
  public void confirm(Order o) {
    System.out.println(
      "Already confirmed"); }
  public void startPreparing(Order o) {
    System.out.println(
      "Already cooked"); }
  public void dispatch(Order o) {
    System.out.println(
      "Already on the way"); }
}
```

1.4 *Draw a class diagram showing how QuickBite connects to two restaurants' point-of-sale systems.* 5 marks



Target interface owned by QuickBite; one adapter per vendor implements it and holds the vendor's class (the adaptee).

1.5 *Give one challenge of the "Place order" design in (c) and how you would overcome it.* 3 marks

Challenge: if one service fails half way (payment succeeded but no rider is free), the order is left half done, and the facade becomes a single point of failure. **Fix:** compensating actions inside the facade (refund the payment, tell the restaurant to cancel), timeouts and retries for each service, and run the facade as several stateless copies behind a load balancer. (Other valid answer: the facade growing into a God class → split it into OrderFacade and TrackingFacade, delegating only.)

2 Paper 1 · Question 2

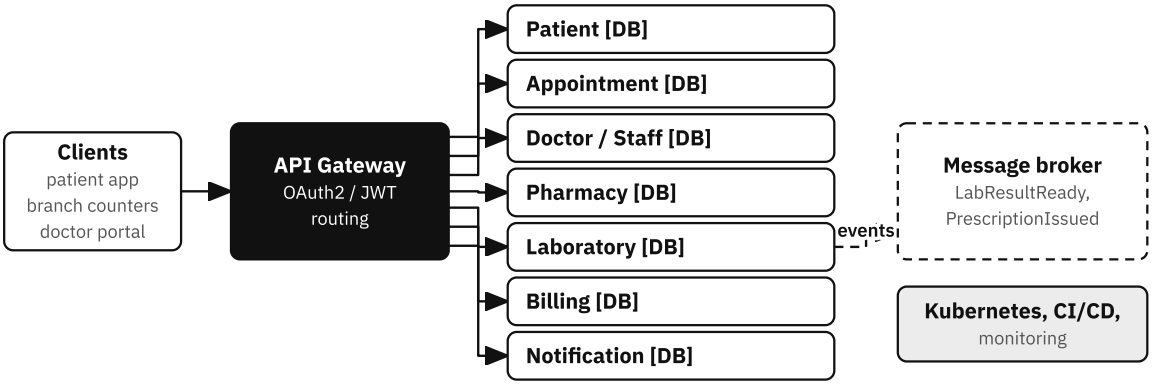
Hospital management system

PRACTICE PAPER 1
25 MARKS

QUESTION 2 · 25 MARKS

A private hospital group with five branches wants a new system built as **microservices**. Key functional requirements: **Patient management** (register patients, medical history, profiles) · **Appointments** (book, reschedule, cancel channelling sessions with doctors) · **Doctors & staff** (schedules, availability, specialities) · **Pharmacy** (prescriptions, stock, dispensing) · **Laboratory** (test orders and results) · **Billing** (invoices, insurance claims, payments) · **Notifications** (SMS reminders, results ready).

2.1 *Draw a diagram to depict the microservices scenario.* 5 marks



2.2 *How can the hospital implement and manage this architecture to achieve scalability, fault isolation and continuous deployment?* 5 marks

Scalability	Scale each service on its own: **Appointments** peaks in the morning when channelling opens, so it auto-scales; Lab and Pharmacy scale per branch; Billing stays small. Stateless services on Kubernetes with auto-scaling and caching of doctor schedules.
Fault isolation	Each service has its own process and database. If Notification fails, appointments are still booked and SMS messages wait in the queue; if Billing is down, doctors still see patients. Circuit breakers, timeouts, retries, bulkheads.
Continuous deployment	A repository and CI/CD pipeline per service, automated tests, Docker images, blue-green releases at night, versioned APIs, so Pharmacy can be updated without touching Appointments.
Manage it	API gateway, service registry, config server, central logging, metrics and tracing; one team per service.

2.3 *Propose a suitable microservices architecture.* 5 marks

One service per capability, each with its **own database**: **Patient**, **Appointment**, **Doctor/Staff**, **Pharmacy**, **Laboratory**, **Billing** (with an insurance adapter), **Notification**. Web portal, mobile app and branch counters go through an **API gateway** (authentication, routing). Services use REST when they need an answer now (Appointment asks Doctor "is Dr. Perera free at 10:00?") and publish events to a **message broker** otherwise ("LabResultReady" → Notification sends an SMS; "PrescriptionIssued" → Pharmacy prepares it). Containers on Kubernetes across the branches, plus a service registry, config server and monitoring.

2 Paper 1 · Question 2 (cont.)

continued

PRACTICE PAPER 1
25 MARKS

2.4 *What methods can maintain data consistency across the services?* 5 marks

- **Database per service**, so no shared transactions; accept **eventual consistency** where safe (notification history).
- **Saga** for an appointment: reserve the doctor's slot → take the booking fee → confirm. If payment fails, a **compensating action** frees the slot.
- **Events** through a broker with a **transactional outbox** and **idempotent consumers**, so a repeated "PrescriptionIssued" doesn't dispense twice.
- **CQRS** read models for a patient's full history view; stronger consistency for medication stock and billing.

2.5 *How would you secure patient data and communication between services?* 5 marks

- **Users:** OAuth2 / OpenID Connect at the gateway with JWT; strict roles (doctor, nurse, pharmacist, cashier, patient), so a cashier can't read lab results.
- **Patient data:** encrypted at rest and in transit, access to records logged (audit trail), data kept to health-data privacy rules, secrets in a vault.
- **Service to service:** mutual TLS (service mesh), short-lived tokens, least-privilege database users per service.
- Rate limiting, input validation, a web application firewall, and alerts on unusual access (one user opening hundreds of records).

3 Paper 1 · Question 3

Smart home controller

PRACTICE PAPER 1
25 MARKS

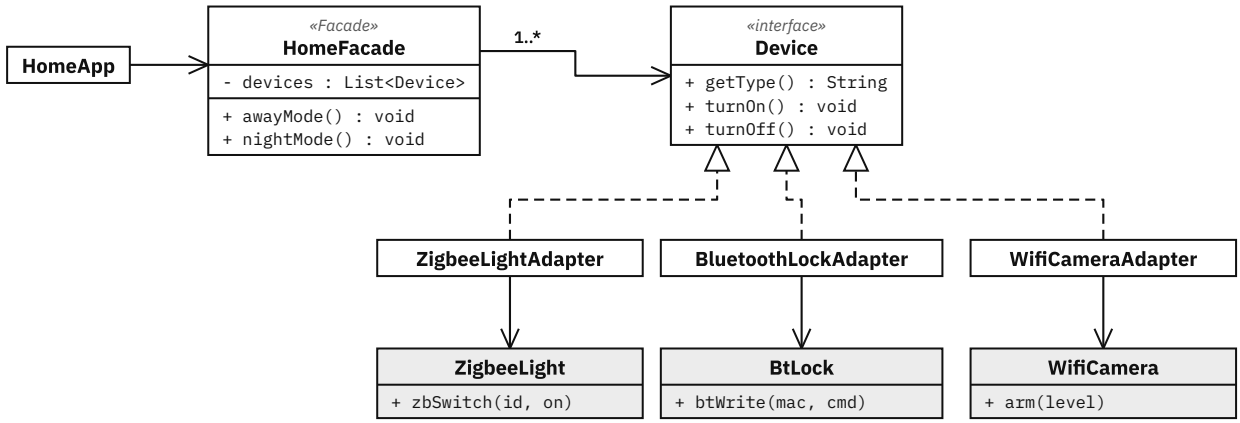
QUESTION 3 · 25 MARKS

A company builds a **smart home app**. Homes have lights, door locks, air conditioners and cameras from different manufacturers; some speak Zigbee (`zbSwitch(id, on)`), others Bluetooth (`btWrite(mac, cmd)`), and the app can't change the devices' software.
An **"Away mode"** button must turn off all lights, lock every door, set the AC to eco and arm the cameras, all in one tap.
Family **members** can control everything remotely, but **guests** may only switch lights; every remote command must be checked and logged.
Every phone in the house must talk to **one home hub controller**; two controllers would send conflicting commands to the same device.

3.1 *Identify the design patterns, with reasons, for: (a) devices with different protocols, (b) the "Away mode" button, (c) member vs guest control, (d) the one home hub controller.* — 8 marks

(a) different protocols	**Adapter**	The app talks to one `Device` interface (`turnOn()`, `turnOff()`); `ZigbeeLightAdapter` and `BluetoothLockAdapter` translate to the vendors' calls, which can't be changed.
(b) Away mode	**Facade**	`HomeFacade.awayMode()` runs four subsystems in order behind one call, so the app doesn't coordinate lights, locks, AC and cameras itself.
(c) member vs guest	**Protection Proxy**	`DeviceProxy` has the same interface as the device, checks the user's role before forwarding (guests: lights only), and logs every command.
(d) one controller	**Singleton**	`HomeHub.getInstance()` gives every phone the same controller, so commands go through one queue and never conflict.

3.2 *Draw a class diagram showing how "Away mode" works with the different devices.* — 7 marks



Facade on the left, the common Device interface in the middle, adapters realising it, and each adapter pointing to its vendor class.

3 Paper 1 · Question 3 (cont.)

continued

3.3 *Write Java for the component that stops guests controlling anything except lights.* 5 marks

```
public interface Device {
  String getType();       // "LIGHT", "LOCK"…
  void turnOn();
  void turnOff();
}
public class DeviceProxy implements Device {
  private final Device real;
  private final User user;
  public DeviceProxy(Device d, User u) {
    real = d; user = u; }
  public String getType() {
    return real.getType(); }
  public void turnOn() {
    check();
    AuditLog.record(user, real, "ON");
    real.turnOn();
  }
  public void turnOff() {
    check();
    AuditLog.record(user, real, "OFF");
    real.turnOff();
  }
  private void check() {
    if (user.isGuest()
        && !real.getType().equals("LIGHT"))
      throw new AccessDeniedException(
          "Guests can only control lights");
  }
}
```

3.4 *Explain which SOLID principles your design follows.* 5 marks

- **Single Responsibility:** each adapter only translates for one vendor; the proxy only checks access; the facade only coordinates.
- **Open/Closed:** a new device brand is a new adapter, and a new scene ("Night mode") is a new facade method; existing classes don't change.
- **Liskov Substitution:** a `DeviceProxy` or any adapter can be used anywhere a `Device` is expected.
- **Interface Segregation:** `Device` is small; a camera-only action (`arm()`) goes in a separate `Armable` interface rather than forcing lights to implement it.
- **Dependency Inversion:** the facade and hub depend on the `Device` abstraction, not on Zigbee or Bluetooth classes.

4 Paper 1 · Question 4

Online examination platform

PRACTICE PAPER 1
25 MARKS

QUESTION 4 · 25 MARKS

A university's **online exam platform** runs every exam through the same process: **authenticate the student → load the paper → conduct the exam → evaluate → publish results**. Evaluation differs by exam type: MCQ exams are auto-marked, essay exams are marked by a lecturer with a rubric, and programming exams run hidden test cases. Question objects (MCQ, essay, coding) are created depending on what the lecturer adds. When a student submits, the answer script goes through checks in order: **time limit → file format → plagiarism**; more checks will be added later. Departments may switch on optional proctoring features (webcam monitoring, screen recording, AI flagging) for any exam, in any combination.

4.1 *Identify suitable patterns, with reasons, for: (a) the common exam process, (b) creating question objects, (c) submission checks, (d) optional proctoring features.* — 8 marks

(a) exam process	**Template Method**	The five steps are fixed in a final `runExam()`; only `evaluate()` differs, so the order is enforced and the common steps are written once.
(b) question objects	**Factory Method**	`QuestionFactory.create(type)` returns MCQ, Essay or Coding questions, so the editor never uses `new` and new types are easy to add.
(c) submission checks	**Chain of Responsibility**	Time limit → format → plagiarism as linked handlers; any failure stops it; more checks are one class and one link each.
(d) proctoring	**Decorator**	Each feature wraps the exam session and adds its monitoring, in any combination at runtime, without a subclass per mix.

4.2 *Write the abstract class for the exam process using the pattern from (a).* — 5 marks

```
public abstract class ExamProcess {
    public final void runExam(Student s) {      // template method: order is fixed
        if (!authenticate(s)) return;
        Paper p = loadPaper();
        Script script = conduct(s, p);
        double mark = evaluate(script);             // the step that differs
        publish(s, mark);
    }
    private boolean authenticate(Student s) { return Auth.verify(s); }   // common
    private Paper loadPaper() { return PaperStore.current(); }       // common
    protected Script conduct(Student s, Paper p) { return Exam.run(s, p); } // hook
    protected abstract double evaluate(Script script);
    private void publish(Student s, double m) { Results.save(s, m); }  // common
}
```

4.3 *Write a ProgrammingExam subclass.* — 3 marks

```
public class ProgrammingExam extends ExamProcess {
    protected double evaluate(Script script) {
        int passed = TestRunner.runHiddenTests(script.getCode());
        return passed * 100.0 / TestRunner.TOTAL;      // percentage of tests passed
    }
}
```

4 Paper 1 · Question 4 (cont.)

continued

PRACTICE PAPER 1
25 MARKS

4.4 *Outline how the submission checks would be implemented so new checks can be added easily.* 5 marks

An abstract `SubmissionCheck` holds a `next` link and a `check(script)` method. `TimeLimitCheck`, `FormatCheck` and `PlagiarismCheck` each test one thing. The chain is built once (cheap checks first, the expensive plagiarism scan last) and every submission goes to the first check. A failure stops the chain with a reason ("submitted 3 minutes late"); passing all means the script is accepted. A new rule (e.g. `WordCountCheck`) is one new class added with one link; nothing else changes.

```
abstract class SubmissionCheck {
  private SubmissionCheck next;
  SubmissionCheck then(SubmissionCheck n) {
    next = n; return n; }
  boolean check(Script s) {
    if (!test(s)) return false;
    return next == null || next.check(s);
  }
  abstract boolean test(Script s);
}
SubmissionCheck c = new TimeLimitCheck();
c.then(new FormatCheck())
 .then(new PlagiarismCheck());
```

4.5 *Explain how the proctoring features can be added without changing the existing exam classes.* 4 marks

Use the **Decorator** pattern. An `ExamSession` interface (`start()`, `end()`) is implemented by `BasicExamSession`. An abstract `ProctoringDecorator` implements `ExamSession` and holds the session it wraps. `WebcamMonitor`, `ScreenRecorder` and `AiFlagging` each add their own behaviour in `start()` / `end()` and then call the wrapped session. A department that wants webcam + AI flagging gets `new AiFlagging(new WebcamMonitor(new BasicExamSession()))`. No exam class changes, and a new feature is one new decorator (Open/Closed).

1 Paper 2 · Question 1

Ride-hailing app

PRACTICE PAPER 2
25 MARKS

PRACTICE PAPER 2

INTE 22313 – Software Design Patterns and Frameworks · No. of questions: 4 · Time: 2½ hours
Answer ALL questions. Allocate time by the marks. State any assumptions you make. This is an OPEN BOOK exam.

QUESTION 1 · 25 MARKS

GoLanka is a ride-hailing app. A trip goes from **Requested → Driver assigned → In progress → Completed**. A trip can be **Cancelled** while Requested or Driver assigned, but not once it is in progress. A passenger chooses a vehicle type when booking (**Tuk**, **Car** or **Van**), and each type has its own base fare and capacity.
The final fare is the base fare plus any extras that apply to that trip, in any combination: **surge pricing**, **highway toll**, **night charge**, **extra luggage**. The company does not want a class for every combination.
Surge multipliers and night rates are changed by the operations team during the day, and every part of the system must use the **same current values**.

1.1 *Identify suitable design patterns, with reasons, for: (a) trip stages, (b) creating vehicle types, (c) fare extras, (d) the shared current rates.* — 8 marks

(a) trip stages	**State**	Cancel is allowed only in the first two stages, and "start trip" only after a driver is assigned. Each stage is a class with its own rules and next stage, so there is no if/else on a status field.
(b) vehicle types	**Factory Method**	`VehicleFactory.create(type)` returns a Tuk, Car or Van from the passenger's choice; booking code never uses `new`, and adding "Bike" is one class.
(c) fare extras	**Decorator**	Each extra wraps the fare and adds its amount, so any combination is built per trip at runtime without $2^4 = 16$ subclasses.
(d) shared rates	**Singleton**	One `FareConfig` instance holds the current surge and night rates, so every service reads the same values; an update applies everywhere at once.

1.2 *Define a Fare interface with getAmount() and getBreakdown(), a BaseFare class, and one extra.* — 6 marks

```
public interface Fare {
  double getAmount();
  String getBreakdown();
}
public class BaseFare implements Fare {
  private final Vehicle v;
  private final double km;
  public BaseFare(Vehicle v, double km) {
    this.v = v; this.km = km; }
  public double getAmount() {
    return v.getBaseRate() * km; }
  public String getBreakdown() {
    return v.getType() + " " + km + "km"; }
}
```

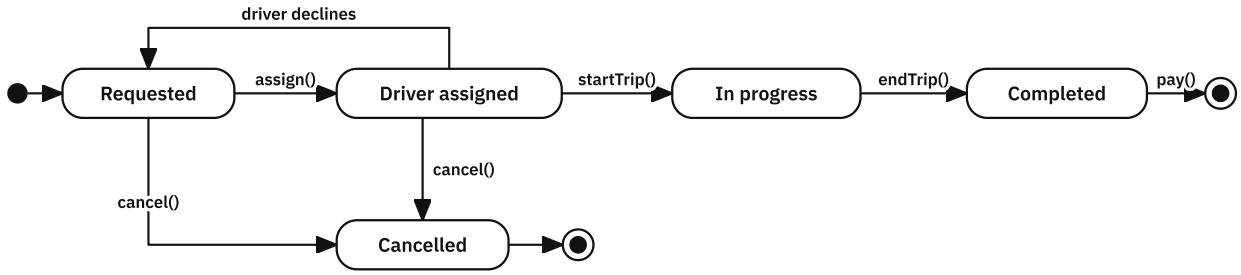
```
public abstract class FareExtra
        implements Fare {
  protected final Fare fare;   // wrapped
  public FareExtra(Fare f) { fare = f; }
}
public class NightCharge extends FareExtra {
  public NightCharge(Fare f) { super(f); }
  public double getAmount() {
    return fare.getAmount() * (1 +
      FareConfig.getInstance().nightRate());
  }
  public String getBreakdown() {
    return fare.getBreakdown()
           + " + night"; }
}
Fare f = new NightCharge(new Toll(
           new BaseFare(car, 12)));
```

1 Paper 2 · Question 1 (cont.)

continued

PRACTICE PAPER 2
25 MARKS

1.3 *Draw a state diagram for a trip.* 5 marks



1.4 *Explain how the shared rates object is created and used safely when many requests arrive at the same time.* 6 marks

`FareConfig` is a **Singleton**: private constructor, a private static `volatile` instance, and a public static `getInstance()` using **double-checked locking**, so two threads arriving at once can't create two objects. The rates themselves are held in thread-safe fields (`volatile`, or an `AtomicReference` to an immutable `Rates` object), so a reader never sees half an update. The operations team updates the rates through `update()`, which swaps in a new object in one step. Every `SurgeCharge` and `NightCharge` decorator reads `FareConfig.getInstance()` when it calculates, so every fare uses the same current values.

```
public final class FareConfig {
  private static volatile FareConfig inst;
  private volatile Rates rates =
      Rates.loadFromDb();
  private FareConfig() { }
  public static FareConfig getInstance() {
    if (inst == null) {
      synchronized (FareConfig.class) {
        if (inst == null)
          inst = new FareConfig();
      }
    }
    return inst;
  }
  public double nightRate() {
    return rates.night; }
  public void update(Rates r) {
    rates = r; }     // one atomic swap
}
```

2 Paper 2 · Question 2

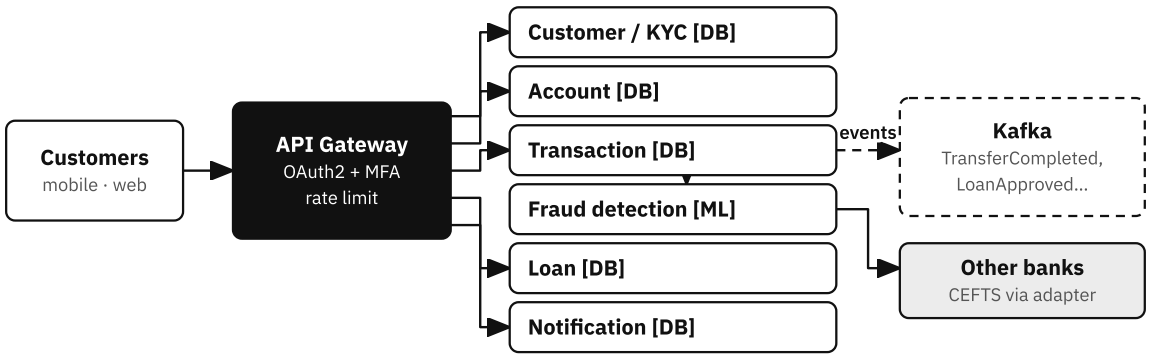
Online banking platform

PRACTICE PAPER 2
25 MARKS

QUESTION 2 · 25 MARKS

A bank is replacing its core online banking system with **microservices**. Key functional requirements: **Customer management** (KYC, profiles, login devices) · **Accounts** (savings, current, fixed deposits, balances) · **Transactions** (transfers between accounts and to other banks, bill payments) · **Loans** (applications, approvals, repayment schedules) · **Fraud detection** (real-time checks on transactions) · **Notifications** (SMS/email alerts for every transaction).

2.1 *Draw a diagram to depict the scenario.* 5 marks



2.2 *How can the bank implement and manage microservices to achieve scalability, fault isolation and continuous deployment?* 5 marks

Scalability	Transactions and Fraud carry the load on salary day and month-end, so they auto-scale independently; Loans stays small. Stateless services on Kubernetes; balance reads cached.
Fault isolation	Own process and database per service. If Notification fails, transfers still work and alerts queue up; if the Loans service fails, payments are unaffected. Circuit breakers, timeouts, bulkheads; a fraud-check timeout falls back to a safe rule (hold large transfers).
Continuous deployment	Pipeline per service with automated and security tests, canary releases to a small share of customers, versioned APIs, quick rollback. Changes are audited for regulators.
Manage it	API gateway, service registry, config server, central logging, metrics and tracing (follow one transfer across services).

2.3 *Propose a suitable microservices architecture.* 5 marks

Services, each with its **own database**: **Customer** (KYC, devices), **Account** (balances, account types), **Transaction** (transfers, bill payments; talks to other banks through a CEFTS/SWIFT **adapter**), **Loan**, **Fraud** (real-time scoring, ML), **Notification**. Mobile and web banking go through an **API gateway** with strong authentication. Transaction calls Fraud synchronously (it needs an answer before the money moves) and publishes events to **Kafka** ("TransferCompleted" → Notification sends an SMS; Account updates statements). Kubernetes, service mesh, registry, config and monitoring, deployed across two data centres.

2 Paper 2 · Question 2 (cont.)

continued

2.4 *How would you maintain data consistency, especially for money transfers?* 5 marks

- **Database per service**, so no distributed transactions across services by default.
- **Saga** for a transfer: debit the sender → credit the receiver → record the transaction. If the credit fails, a **compensating action** reverses the debit. An **orchestrator** in the Transaction service tracks each step.
- **Idempotency keys** on every transfer request, so a retried request never debits twice; a **transactional outbox** for events.
- Money movements inside one account database use normal ACID transactions; balances are **strongly consistent**, while alerts and statements are eventually consistent.
- Daily **reconciliation** jobs to catch any mismatch.

2.5 *How would you secure customer data and service-to-service communication?* 5 marks

- **Customers:** OAuth2 / OpenID Connect with **multi-factor authentication** (OTP), device binding, short-lived JWTs, automatic logout.
- **Data:** encryption at rest and in transit, card data to **PCI-DSS**, keys in a hardware security module or vault, data masking in logs.
- **Service to service:** mutual TLS through a service mesh, least privilege per service (Notification can't read balances).
- **Monitoring:** rate limits, a WAF, fraud alerts, full audit trail for regulators.

3 Paper 2 · Question 3

Insurance claim processing

PRACTICE PAPER 2
25 MARKS

QUESTION 3 · 25 MARKS

An insurance company approves claims by amount: an **Agent** can approve up to Rs. 50,000, a **Supervisor** up to Rs. 500,000, a **Manager** up to Rs. 2,000,000, and anything above goes to the **Claims Board**. New approval levels may be added.

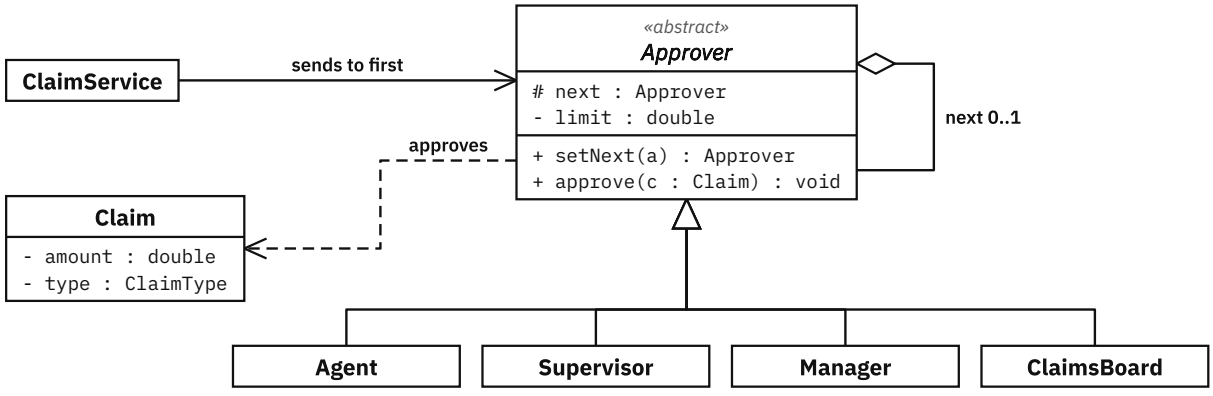
Every claim is processed in the same order: **register → assess → approve → pay → notify**. Assessment differs for vehicle, health and property claims (a vehicle claim needs a garage estimate; a health claim needs hospital bills checked; a property claim needs a site visit).

Payments are made through the bank's old mainframe system, which only offers `TRNPAY(String acctNo, long cents, String refCode)`. The claims system expects `pay(Claim c)`.

3.1 *Identify the design patterns, with reasons, for: (a) approval by amount, (b) the processing order, (c) the mainframe payment.* — 6 marks

(a) approval levels	**Chain of Responsibility**	The claim is passed up Agent → Supervisor → Manager → Board until a level with a high enough limit approves it; the sender doesn't need to know who approves, and new levels are new handlers.
(b) processing order	**Template Method**	A final `process()` fixes register → assess → approve → pay → notify; only `assess()` differs by claim type, so the order is enforced and common steps are written once.
(c) mainframe	**Adapter**	`MainframePaymentAdapter` implements `PaymentService.pay(Claim)` and converts it into `TRNPAY(acct, cents, ref)`; the mainframe can't be changed.

3.2 *Draw a class diagram for the approval levels.* — 6 marks



3 Paper 2 · Question 3 (cont.)

continued

PRACTICE PAPER 2
25 MARKS

3.3 *Write Java showing how a claim of Rs. 750,000 is processed through the approval levels.* 6 marks

```
public abstract class Approver {
  protected Approver next;
  private final double limit;
  protected Approver(double limit) {
    this.limit = limit; }
  public Approver setNext(Approver n) {
    next = n; return n; }

  public void approve(Claim c) {
    if (c.getAmount() <= limit) {
      System.out.println(getClass()
        .getSimpleName() + " approved "
        + c.getAmount());
    } else if (next != null) {
      next.approve(c);     // pass it up
    }
  }
}
```

```
class Agent extends Approver {
  Agent() { super(50_000); } }
class Supervisor extends Approver {
  Supervisor() { super(500_000); } }
class Manager extends Approver {
  Manager() { super(2_000_000); } }
class ClaimsBoard extends Approver {
  ClaimsBoard() {
    super(Double.MAX_VALUE); } } // fallback

// client
Approver chain = new Agent();
chain.setNext(new Supervisor())
     .setNext(new Manager())
     .setNext(new ClaimsBoard());
chain.approve(new Claim(750_000));
// Agent: no → Supervisor: no
// → "Manager approved 750000.0"
```

3.4 *Every approval level must log the claim, record its decision and log the result. How would you enforce this?* 3 marks

Combine **Template Method** with the chain: make `approve()` in `Approver` **final** and have it call `logReceived()` → `decide(c)` (abstract, written by each level) → `logResult()`, then pass the claim on if not approved. Levels only write `decide()`, so none can skip the logging.

3.5 *Show how the claims system can pay through the mainframe without changing either system.* 4 marks

Adapter. `PaymentService` is the target interface the claims system already uses. `MainframePaymentAdapter` implements it, holds the mainframe client (adaptee), and translates: rupees → cents, claim → account number and reference code, and the mainframe's return code → an exception on failure. The `pay()` step in the template calls `paymentService.pay(c)` and never sees the mainframe.

```
class MainframePaymentAdapter
      implements PaymentService {
  private final Mainframe mf;
  public void pay(Claim c) {
    long cents =
      Math.round(c.getAmount() * 100);
    int rc = mf.TRNPAY(c.getAccountNo(),
                       cents, c.getRef());
    if (rc != 0) throw new
      PaymentException("code " + rc);
  }
}
```

4 Paper 2 · Question 4

Five short cases

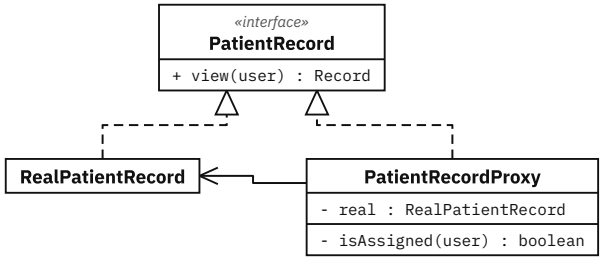
PRACTICE PAPER 2
25 MARKS

QUESTION 4 · 25 MARKS

For each case, explain the design pattern you would suggest and why, and give a simple class diagram or code outline.

4.1 *A hospital system: patient records must only be opened by doctors and nurses assigned to that patient, and every access must be recorded.* 5 marks

Protection Proxy (structural). `PatientRecordProxy` implements the same `PatientRecord` interface as the real record. `view()` first checks the user's role and whether they are assigned to the patient, writes an audit entry, and only then forwards to the real record; otherwise it refuses. **Reasons:** sensitive data is never reached directly; the security rule is in one place; the record class stays free of security code.



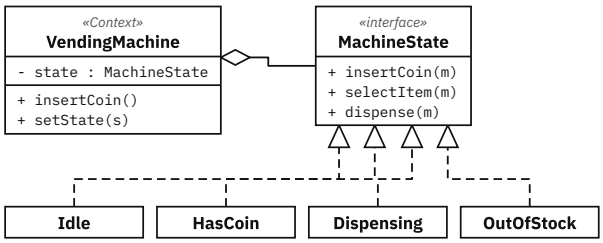
4.2 *A reporting module exports the same report as PDF, Excel or CSV, depending on what the user selects; more formats will be added.* 5 marks

Factory Method (creational). A `ReportExporter` interface with `export(report)`; `PdfExporter`, `ExcelExporter` and `CsvExporter` implement it. `ExporterFactory.create(format)` returns the right one from the user's choice. **Reasons:** the report screen depends only on the interface, creation logic is in one place, and a new format (JSON) is one new class.

```
class ExporterFactory {
  static ReportExporter create(String f) {
    switch (f) {
      case "PDF":   return new PdfExporter();
      case "EXCEL":
        return new ExcelExporter();
      case "CSV":   return new CsvExporter();
      default: throw new
        IllegalArgumentException(f);
    }
  }
}
```

4.3 *A vending machine behaves differently when it is idle, has a coin inserted, is dispensing, or is out of stock.* 5 marks

State (behavioural). `VendingMachine` (context) holds a `MachineState`; `IdleState`, `HasCoinState`, `DispensingState` and `OutOfStockState` implement `insertCoin()`, `selectItem()`, `dispense()` and `refund()`. Inserting a coin in Idle moves to HasCoin; in OutOfStock the coin is refunded. **Reasons:** the same button behaves differently by state, the if/else on a status flag disappears, and each state controls its own transitions. Stateless states can be shared Singletons.



4 Paper 2 · Question 4 (cont.)

continued

PRACTICE PAPER 2
25 MARKS

4.4 *An application sends SMS through an old gateway class with sendText(number, text). A new provider offers dispatch(Message m, int priority). Hundreds of calls to sendText() exist.* 5 marks

Adapter (structural). Keep `SmsGateway` with `sendText()` as the target. `NewProviderAdapter` implements it, holds the new provider (adaptee), builds a `Message` from the number and text, and calls `dispatch(m, NORMAL)`. **Reasons:** the hundreds of existing calls don't change (no risk to tested code), the provider's API isn't ours to change, and switching providers later means one new adapter.

```
class NewProviderAdapter
      implements SmsGateway {
  private final NewSmsProvider p;
  NewProviderAdapter(NewSmsProvider p) {
    this.p = p; }
  public void sendText(String no,
                       String text) {
    p.dispatch(new Message(no, text), 1);
  }
}
```

4.5 *A pizza shop: customers choose a base pizza and add toppings (cheese, chicken, olives, extra sauce) in any combination; the price must be calculated automatically.* 5 marks

Decorator (structural). `Pizza` interface with `getPrice()` and `getDescription()`; `PlainPizza` is the base; `Cheese`, `Chicken`, `Olives` and `ExtraSauce` extend an abstract `ToppingDecorator` that holds a `Pizza`. Each topping adds its price and passes the rest of the call on. **Reasons:** no subclass per combination, toppings are picked at runtime, and new toppings don't change existing classes.

```
Pizza p = new PlainPizza();     // 1200
p = new Cheese(p);              // +300
p = new Chicken(p);             // +450
p = new Olives(p);              // +150
p.getPrice();                    // 2100

class Cheese extends ToppingDecorator {
  Cheese(Pizza p) { super(p); }
  public double getPrice() {
    return pizza.getPrice() + 300; }
}
```

1 Paper 3 · Question 1

EV charging station

PRACTICE PAPER 3
25 MARKS

PRACTICE PAPER 3

INTE 22313 – Software Design Patterns and Frameworks · No. of questions: 4 · Time: 2½ hours
Answer ALL questions. Allocate time by the marks. State any assumptions you make. This is an OPEN BOOK exam.

QUESTION 1 · 25 MARKS

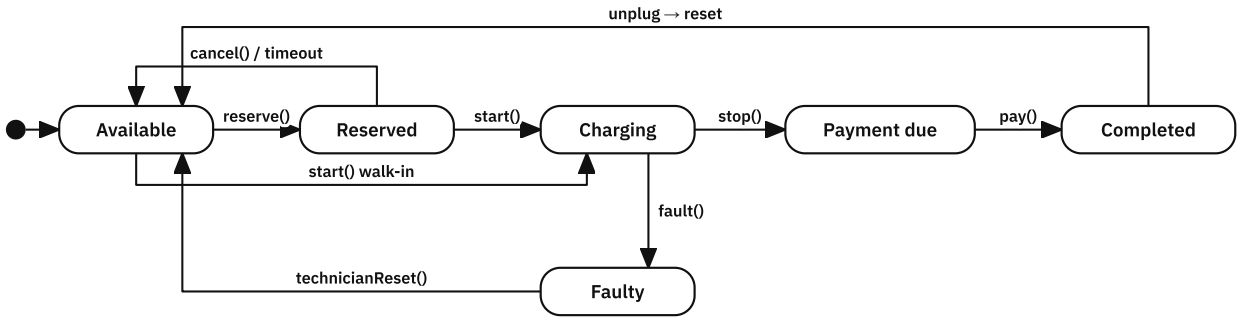
A city installs **electric vehicle (EV) charging points**. A charging session can be in these states: **Available**, **Reserved** (a driver booked it in the app), **Charging**, **Payment due** and **Completed**. Any point can also go **Faulty**, and then it accepts nothing until a technician resets it. What the point does when a driver taps "start", "stop" or "pay" depends on its current state.

The charging hardware comes from two vendors that the city cannot modify: Vendor A's charger offers `startCharge(double kW)` / `halt()`; Vendor B's offers `beginSession(int port, String mode)` / `endSession(int port)`. The session software expects `start()` and `stop()`.

1.1 *Explain why the State pattern is appropriate for a charging session.* 4 marks

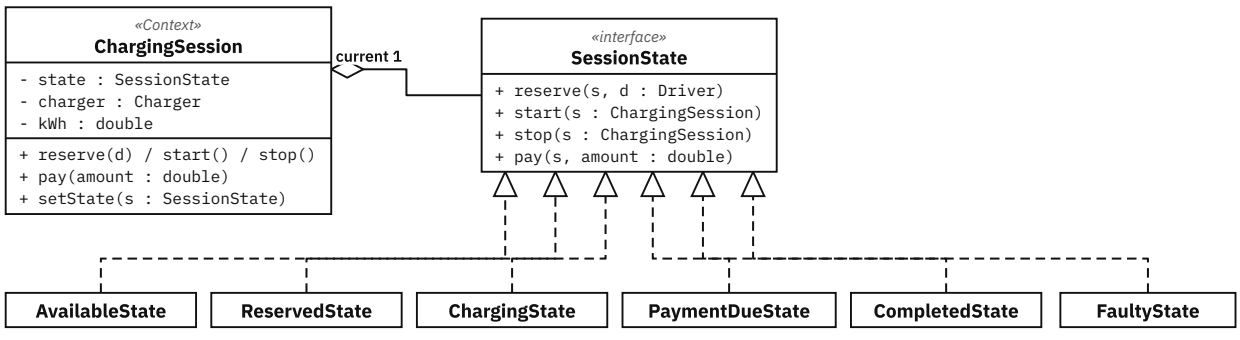
A session has a **fixed set of well-defined states**, and the **same action behaves differently in each**: "start" begins charging when Reserved (for that driver) but is refused when Faulty; "pay" only works in Payment due. Without the pattern, every method needs a growing if/switch on the status. With State, each state is a class holding its own rules and deciding the next state, so invalid actions are refused cleanly, transitions are explicit, and a new state (e.g. *Overstay* with an idle fee) is just a new class.

1.2 *Draw a state diagram for a charging session.* 5 marks



"Any state → Faulty" can be drawn as one note, or arrows from the main states, labelled fault().

1.3 *Design a UML class diagram for the charging session using the State pattern.* 6 marks



1 Paper 3 · Question 1 (cont.)

continued

PRACTICE PAPER 3
25 MARKS

1.4 *Write Java for the ChargingSession context class.* 4 marks

```
public class ChargingSession {
    private SessionState state = new AvailableState();
    private final Charger charger;          // the adapter from 1.5
    private double kWh;

    public ChargingSession(Charger c) { charger = c; }
    public void setState(SessionState s) { state = s; }
    public Charger getCharger() { return charger; }

    // every request is handed to the current state
    public void reserve(Driver d) { state.reserve(this, d); }
    public void start()           { state.start(this); }
    public void stop()            { state.stop(this); }
    public void pay(double amt)    { state.pay(this, amt); }
    public void fault()           { setState(new FaultyState()); }
}
```

1.5 *The session software must work with both vendors' chargers. Provide a solution using design pattern knowledge, with code for one vendor.* 6 marks

Use the **Adapter** pattern. The session software depends on one target interface it owns, `Charger` with `start()` and `stop()`. `VendorAAdapter` and `VendorBAdapter` implement `Charger`, each holding that vendor's class (the adaptee) and translating the call: Vendor A needs a power level in kW; Vendor B needs a port and a mode. `ChargingState` just calls `session.getCharger().start()` and never knows the vendor. **Why:** the hardware software can't be modified, the state classes stay the same for every vendor, and a third vendor is one more adapter (Open/Closed).

```
public interface Charger {     // target
  void start();
  void stop();
}
public class VendorBAdapter
        implements Charger {
  private final VendorBCharger hw;
  private final int port;
  public VendorBAdapter(VendorBCharger hw,
                        int port) {
    this.hw = hw; this.port = port; }
  public void start() {
    hw.beginSession(port, "FAST"); }
  public void stop() {
    hw.endSession(port); }
}
```

2 Paper 3 · Question 2

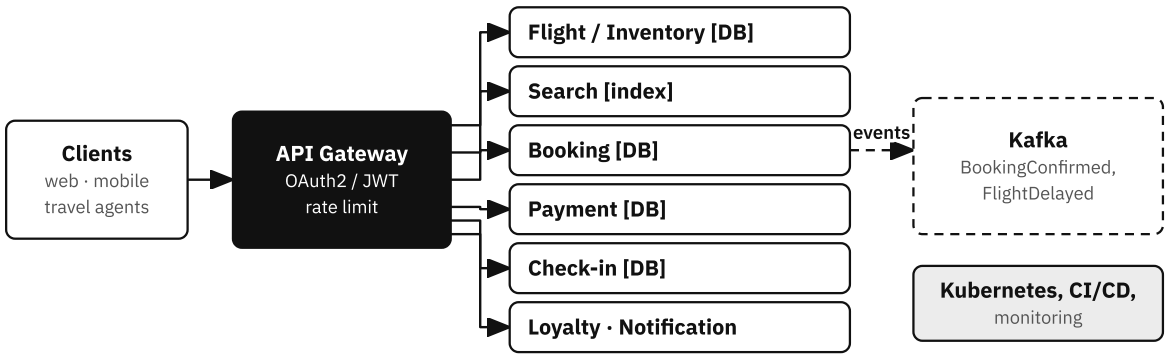
Airline reservation system

PRACTICE PAPER 3
25 MARKS

QUESTION 2 · 25 MARKS

A regional airline is rebuilding its reservation platform as **microservices**. Key functional requirements: **Flight search** (routes, schedules, fares, seat availability) · **Booking** (reserve seats, passenger details, changes and cancellations) · **Payments** (cards, wallets, refunds) · **Check-in** (online check-in, seat selection, boarding passes) · **Loyalty** (miles, tiers, rewards) · **Notifications** (e-tickets, delays, gate changes).

2.1 *Draw a diagram to depict the microservices scenario.* 5 marks



2.2 *How can the airline achieve scalability, fault isolation and continuous deployment?* 5 marks

Scalability	**Flight Search** takes far more traffic than booking (many searches per booking, spikes during sales), so it scales out on its own with a cache of fares and availability; Check-in scales in the hours before departures; Loyalty stays small.
Fault isolation	Own process and database per service. If Loyalty is down, bookings still complete and miles are added later from queued events; if Notification fails, the e-ticket is resent when it recovers. Circuit breakers, timeouts, retries, bulkheads.
Continuous deployment	Pipeline per service with automated tests, Docker images, canary releases (new fare engine to 5% of searches), versioned APIs, quick rollback.
Manage it	API gateway, registry, config server, central logging, metrics and tracing; one team per service.

2.3 *Propose a suitable microservices architecture.* 5 marks

Services, each with its **own database**: **Flight/Inventory** (schedules, seats), **Search** (read-optimised index built from flight events, CQRS), **Booking**, **Payment** (gateway adapters for cards and wallets), **Check-in** (boarding passes), **Loyalty**, **Notification**. Web, mobile and travel-agent clients go through an **API gateway**. Booking calls Flight/Inventory synchronously to hold seats, and publishes events to **Kafka** ("BookingConfirmed" → Loyalty adds miles, Notification sends the e-ticket; "FlightDelayed" → Notification alerts passengers). Kubernetes, registry, config and monitoring.

2.4 *How would you keep data consistent, for example so that two passengers can't get the same seat?* 5 marks

- **Database per service**; the Flight/Inventory service is the **single owner of seat data** and holds seats with a local ACID transaction or an optimistic lock (version number), so the same seat can't be sold twice.
- **Saga** for booking: hold seat (with a 15-minute expiry) → take payment → confirm booking → issue ticket. If payment fails, a **compensating action** releases the seat.
- **Transactional outbox** and **idempotent consumers**, so a repeated "PaymentSucceeded" never books twice.
- **Eventual consistency** for search results and loyalty miles, which can lag a few seconds.

2 Paper 3 · Question 2 (cont.)

continued

PRACTICE PAPER 3
25 MARKS

2.5 *How would you secure passenger data and communication between services?* 5 marks

- **Users:** OAuth2 / OpenID Connect at the gateway with JWT; roles for passenger, travel agent, airport staff, admin.
- **Data:** passport and payment data encrypted at rest and in transit; card data to **PCI-DSS** through the payment provider; personal data handled under privacy laws; secrets in a vault.
- **Service to service:** mutual TLS via a service mesh, least-privilege access (Loyalty can't read passport numbers).
- Rate limiting and bot protection on search, a WAF, audit logs and alerts.

3 Paper 3 · Question 3

Hotel booking system

PRACTICE PAPER 3
25 MARKS

QUESTION 3 · 25 MARKS

A beach hotel builds a booking system. Rooms are **Standard**, **Deluxe** or **Suite**, and the right room object must be created from the guest's choice.
Guests can add optional extras to a room: **breakfast**, **airport pickup**, **late checkout** and **sea view**, in any combination. The price and description must update automatically, without a class for every combination.
The hotel also sells **holiday packages**. Booking a package must reserve the room, book a spa slot, reserve a restaurant table and schedule the airport transfer. The front-desk screen should do this with one call.

3.1 *Identify suitable design patterns, with reasons, for: (a) creating room types, (b) optional extras, (c) booking a package.* — 6 marks

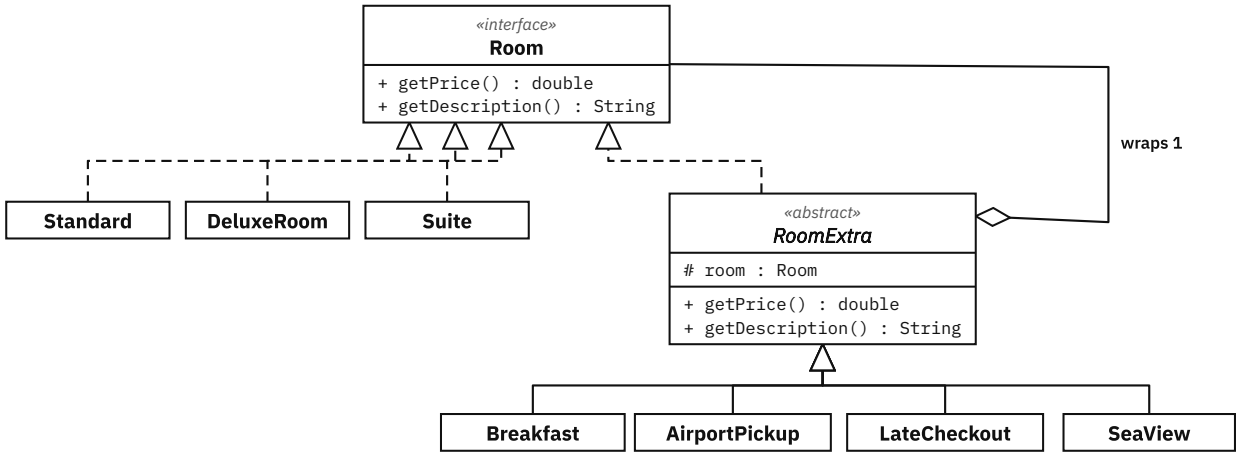
(a) room types	**Factory Method**	`RoomFactory.create(type)` builds a Standard, Deluxe or Suite room from the guest's choice, so booking code uses the `Room` interface only, and a new type (Villa) is one class.
(b) extras	**Decorator**	Each extra wraps the room and adds its own price and description, so any of the $2^4 = 16$ combinations is built at runtime without subclasses.
(c) packages	**Facade**	`PackageFacade.bookPackage()` calls the room, spa, restaurant and transfer services in order behind one method, so the front desk depends on one class.

3.2 *Define a Room interface with getPrice() and getDescription(), and a DeluxeRoom class.* — 5 marks

```
public interface Room {
    double getPrice();      // per night
    String getDescription();
}
```

```
public class DeluxeRoom implements Room {
    private final int number;
    public DeluxeRoom(int number) {
        this.number = number; }
    public double getPrice() {
        return 25000; }
    public String getDescription() {
        return "Deluxe room " + number; }
}
```

3.3 *Draw a class diagram showing how the optional extras are applied.* — 6 marks



`Room r = new LateCheckout(new Breakfast(new DeluxeRoom(204)));` → 25,000 + 3,000 + 4,000.

3 Paper 3 · Question 3 (cont.)

continued

PRACTICE PAPER 3
25 MARKS

3.4 *Write an outline of the class that books a package with one call.* 4 marks

```
public class PackageFacade {
    private final RoomService rooms = new RoomService();
    private final SpaService spa = new SpaService();
    private final RestaurantService dining = new RestaurantService();
    private final TransferService transfer = new TransferService();

    public Booking bookPackage(Guest g, Room room, DateRange dates) {
        Booking b = rooms.reserve(g, room, dates);       // calls each subsystem in order
        spa.bookSlot(g, dates.firstDay());
        dining.reserveTable(g, dates.firstDay());
        transfer.schedule(g, dates.arrival());
        return b;
    }
}
```

3.5 *Discuss two challenges of the design used for the extras and how you would handle them.* 4 marks

- **Many small classes and long wrapping chains** (`new A(new B(new C(room)))`) are hard to read and debug. → A **builder** (or the factory) assembles the chosen extras in one place; each decorator logs what it adds.
- **Order and duplicates matter**: a percentage discount applied before or after extras gives different totals, and breakfast could be added twice. → Fix the wrapping order in the builder and reject duplicate extras.
- *(Also acceptable: identity checks fail because the outer object isn't a DeluxeRoom; performance cost of many layers is small but grows.)*

4 Paper 3 · Question 4

Online store returns and refunds

PRACTICE PAPER 3
25 MARKS

QUESTION 4 · 25 MARKS

An online store automates **returns**. A return request is checked in order: **valid receipt → within 14 days → item condition → refund limit**. The checks must be easy to change and extend.
Every refund follows the same steps: **validate the request → calculate the amount → issue the refund → notify the customer**. Issuing the refund differs for **card**, **cash on delivery** (bank transfer) and **store credit**.
For auditing, every refund must be written to **one central audit log** in order, with no duplicate entries. The returns service must keep working even if a server or a data centre fails.

4.1 *Identify suitable design patterns, with reasons, for: (a) the return checks, (b) the refund steps, (c) the audit log.* 6 marks

(a) return checks	**Chain of Responsibility**	Receipt → 14 days → condition → limit as linked handlers; a failure stops the request with a reason; checks can be reordered or added without touching the others.
(b) refund steps	**Template Method**	A final `processRefund()` fixes the four steps; only `issueRefund()` differs per refund method, so the process is standard and shared steps are written once.
(c) audit log	**Singleton**	One `AuditLogger` instance owns the log and writes entries in order through one synchronized method, so there are no duplicates or interleaving.

4.2 *Write the abstract class for the refund steps, and one subclass.* 5 marks

```
public abstract class RefundProcess {
  public final void processRefund(
          ReturnRequest r) {
    if (!validate(r)) return;
    double amt = calculate(r);
    issueRefund(r, amt);       // varies
    notifyCustomer(r, amt);
    AuditLogger.getInstance()
        .log("Refund " + r.getId()
            + " " + amt);
  }
  protected boolean validate(
          ReturnRequest r) {
    return r.isApproved(); }
  protected double calculate(
          ReturnRequest r) {
    return r.getItemPrice(); }
  protected abstract void issueRefund(
          ReturnRequest r, double a);
  private void notifyCustomer(
          ReturnRequest r, double a) {
    Mailer.send(r.getEmail(),
                "Refunded " + a); }
}
```

```
public class StoreCreditRefund
        extends RefundProcess {
  protected double calculate(
          ReturnRequest r) {
    // 10% bonus for taking store credit
    return r.getItemPrice() * 1.10; }
  protected void issueRefund(
          ReturnRequest r, double a) {
    Wallet.of(r.getCustomer()).add(a); }
}
// CardRefund: gateway.refund(...)
// BankTransferRefund: bank.transfer(...)
```

4 Paper 3 · Question 4 (cont.)

continued

PRACTICE PAPER 3
25 MARKS

4.3 *Outline how the return checks would be implemented.* 5 marks

An abstract `ReturnCheck` holds a link to the next check and a `check(request)` that returns a failure reason or passes the request on. `ReceiptCheck`, `WithinDaysCheck`, `ConditionCheck` and `RefundLimitCheck` each test one thing. The chain is built once (the order can come from configuration) and every request goes to the first check. If all pass, the refund process from 4.2 starts. A new rule, such as `SerialNumberCheck` for electronics, is one new class with one link.

```
abstract class ReturnCheck {
  private ReturnCheck next;
  ReturnCheck then(ReturnCheck n) {
    next = n; return n; }
  String check(ReturnRequest r) {
    String fail = test(r);
    if (fail != null) return fail;
    return next == null ? null
                        : next.check(r);
  }
  abstract String test(ReturnRequest r);
}
class WithinDaysCheck extends ReturnCheck {
  String test(ReturnRequest r) {
    return r.daysSincePurchase() <= 14
        ? null : "Older than 14 days"; }
}
```

4.4 *Write the audit log class and explain why it prevents duplicate or out-of-order entries.* 4 marks

```
public final class AuditLogger {
  private static final AuditLogger
      INSTANCE = new AuditLogger();
  private final AtomicLong seq =
      new AtomicLong();
  private AuditLogger() { }
  public static AuditLogger getInstance() {
    return INSTANCE; }
  public synchronized void log(String e) {
    AuditStore.append(
        seq.incrementAndGet(), e);
  }
}
```

The private constructor means **no other logger object can exist**; every refund calls the same `getInstance()`. Because there is one object, there is one writer and one sequence counter, and `log()` is `synchronized`, so two refunds at the same moment are written one after the other with increasing sequence numbers, never twice and never interleaved. (For several servers, `AuditStore` is a replicated, append-only store.)

4.5 *How would you extend the design for High Availability and Disaster Recovery?* 5 marks

HA: the check chain and refund classes keep no state, so the returns service runs as **several copies behind a load balancer**, with health checks and automatic restart (Kubernetes). Requests go through a **message queue** so none are lost if a copy dies. The database is replicated with automatic failover. Each server's Singleton logger writes to a **shared, replicated audit store**, so the logger isn't a single point of failure, and a refund is made **idempotent** (refund ID checked) so a retry never pays twice. **DR:** regular backups, the database and audit log **replicated to a second region**, agreed RPO/RTO, automated failover, infrastructure-as-code to rebuild, and regular DR drills.